

Assignment 1 - Data wrangling and functions

YENUS IBRAHIM AYALEW

January 15, 2026 | 22:29:19 | CET

```
# LOAD THE PACKAGES YOU ARE USING IN THIS CODE CHUNK
# library(nameofpackage)

library(tidyverse)
```

Task 1: Getting started with the Astronaut Database

The data this week comes from Mariya Stavnichuk and Tatsuya Corlett.

This article talks about the data set in greater detail.

This database contains publically available information about all astronauts who participated in space missions before 15 January 2020 collected from NASA, Roscosmos, and fun-made websites. The provided information includes full astronaut name, sex, date of birth, nationality, military status, a title and year of a selection program, and information about each mission completed by a particular astronaut such as a year, ascend and descend shuttle names, mission and extravehicular activity (EVAs) durations.

Credit for preparing the data set: Georgios Karamanis

You can use the data set directly by importing the csv file in the repository.

If not noted otherwise, for the following tasks, you will work only with a subset of astronauts based on their **year of birth**. The year span you are assigned depends on your first name:

Your first name starts with...	Years of birth
A-I	(1921-1942]
J-R	(1943-1964]
S-Z	(1965-1983]

since my name starts with “Y”

1965 to 1983 will be my assigned year

- a) How many unique astronauts were there in the full data set and how many are there in your subset?
(Note that some might have been part of multiple missions.)

```
# Unique astronuts in the full data set and in my subset
# load the data first
astronuts_data <- read.csv("astronauts.csv")
# column names of the data
colnames(astronuts_data)
```

```
[1] "id" "number"
[3] "nationwide_number" "name"
[5] "original_name" "sex"
[7] "year_of_birth" "nationality"
[9] "military_civilian" "selection"
[11] "year_of_selection" "mission_number"
[13] "total_number_of_missions" "occupation"
[15] "year_of_mission" "mission_title"
[17] "ascend_shuttle" "in_orbit"
[19] "descend_shuttle" "hours_mission"
[21] "field21" "eva_hrs_mission"
[23] "total_eva_hrs"
```

```
# dplyr loading
library(dplyr)
#Unique number of astronauts
astronuts_data %>%
  distinct(name) %>%
  nrow() # This is total number of unique astronuts
```

```
[1] 564
```

```
# For subset give to me
astronuts_data %>%
  filter(year_of_birth >= 1965 & year_of_birth <= 1983) %>%
  distinct(name) %>%
  nrow() # The number of unique astronauts with birth year of 1965-1983
```

```
[1] 87
```

- b) What's the overall share of female astronauts in the subset of data you were assigned to? And what's the share of astronauts with U.S. nationality?

```
# Overall share of female astronauts for subset I have been assigned
my_subset <- astronuts_data %>%
  filter(year_of_birth >= 1965 & year_of_birth <= 1983)
# Calculate female share
female_share <- my_subset %>%
  summarise(
    total_astronauts = n(),
    female_count = sum(sex == "female", na.rm = TRUE),
    female_share = (female_count / total_astronauts) * 100
  )
cat("Share of female astronauts:", round(female_share$female_share, 2), "%\n")
```

Share of female astronauts: 16.91 %

```
Us_nationality <- my_subset %>%
  summarise(
    total_astronauts = n(),
    US_national_count = sum(nationality == "U.S."),
    US_share = (US_national_count/total_astronauts) * 100
  )
cat("Share of US Nationals:", round(Us_nationality$US_share, 2), "%\n")
```

Share of US Nationals: 42.65 %

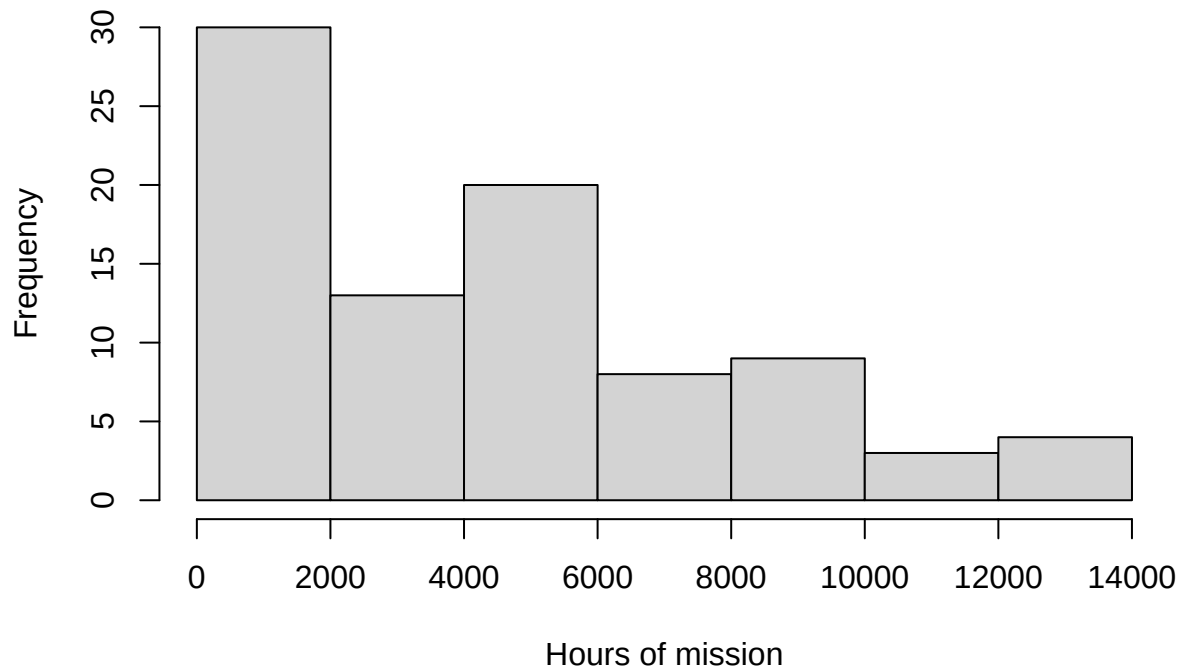
c) Create a new data set `total_hours_df` containing the unique names and the total duration of all missions in hours (`total_hrs_sum`) for each astronaut in the subset. Then, visualize the distribution of that variable using an appropriate plot type.

```
# new data set
total_hours_df <- my_subset %>%
  group_by(name) %>%
  summarise(total_hrs_sum = sum(hours_mission, na.rm = TRUE)) %>%
  ungroup()
head(total_hours_df) #
```

```
# A tibble: 6 x 2
  name                total_hrs_sum
  <chr>                <dbl>
1 Acaba, Joseph M.    7272.
2 Aidyn (Aydyn) Akanovich Aimbetov 236.
3 Al Mansoori, Hazzaa 189
4 Ansari, Anousheh    262.
5 Antonelli, Dominic A. 569
6 Artemyev, Oleg      8784
```

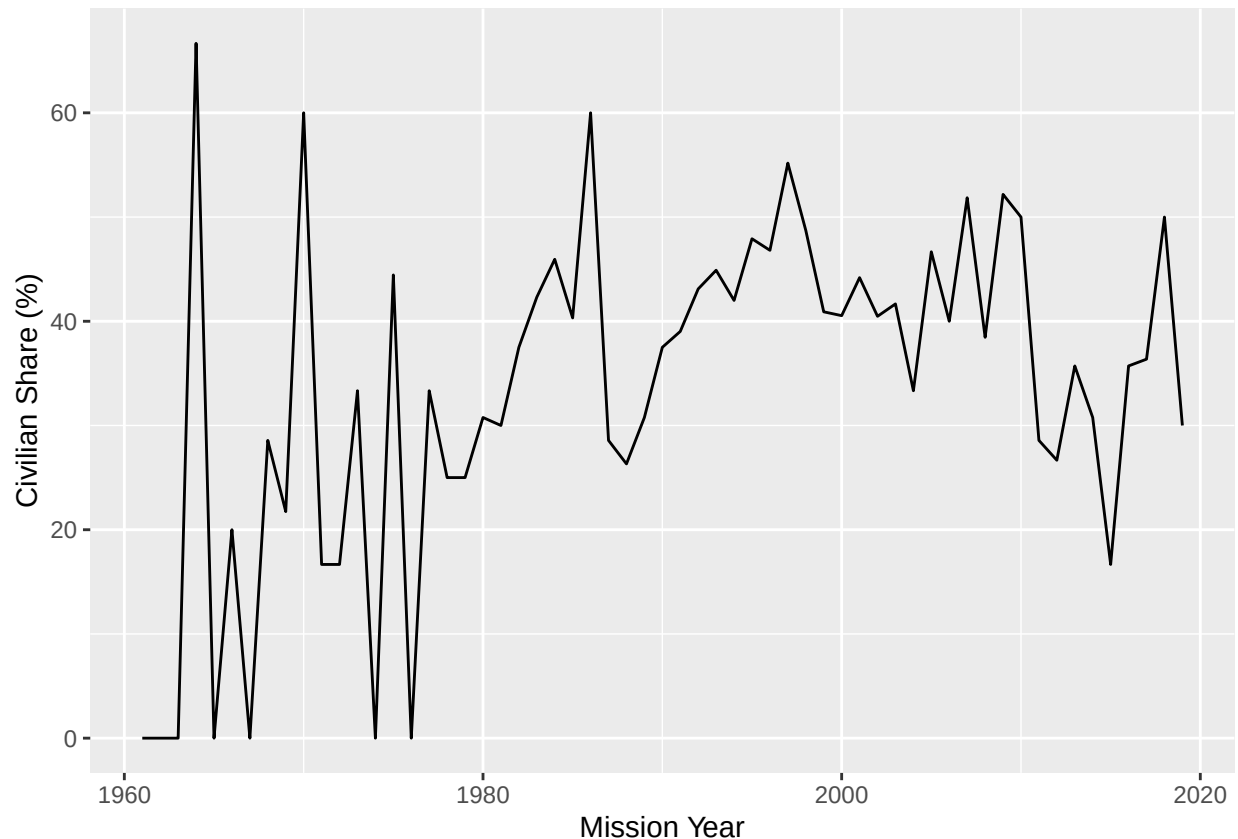
```
# Since it is continuous data we can use histogram to plot it
hist(total_hours_df$total_hrs_sum, xlab = "Hours of mission", main = "Distribution of the total duration")
```

Distribution of the total duration of all missions in hours for each astronaut



d) Using the full data set, provide a plot that shows the overall share of *civilian* astronauts by the year of mission!

```
##
civilian_share <- astronuts_data %>%
  group_by(year_of_mission) %>%
  summarise(civilian_share = mean(military_civilian == "civilian", na.rm = TRUE) * 100)
# Create plot: since it shows trend line graph is more appropriate
ggplot(civilian_share, aes(x = year_of_mission, y = civilian_share)) +
  geom_line() +
  labs(x = "Mission Year", y = "Civilian Share (%)")
```



Task 2 – Silly Password Strength Checker Program a function `check_password(pw)` that takes a password string `pw` and returns a funny strength rating* according to the following rules:

1. If the password contains the word "password" (in any mix of upper/lower case), return "Terrible! ".
2. If the password is shorter than 6 characters, return "Too weak ".
3. If it contains only letters, return "Meh ".
4. If it contains letters and** numbers, return "Better ".
5. If it contains letters **and** numbers **and** at least one special character from `!@#$$%^&*?`, return "Strong ".

Write the function and test it with at least **five different passwords** that trigger different outputs. Also, use functions from the `stringr` package to execute the checks in your function (hint: look up `str_detect()` and `str_length()`).

```
library(stringr)

check_password <- function(pw) {
  # Rule 1: Contains "password" (case-insensitive)
  if (str_detect(pw, regex("password", ignore_case = TRUE))) {
    return("Terrible! ")
  }

  # Rule 2: Too short
  if (str_length(pw) < 6) {
    return("Too weak ")
  }
}
```

```

# Character class checks
has_letter <- str_detect(pw, "[A-Za-z]")
has_number <- str_detect(pw, "[0-9]")
has_special <- str_detect(pw, "[!@#$$%^&*?]")

# Rule 5: letters + numbers + special
if (has_letter && has_number && has_special) {
  return("Strong ")
}

# Rule 4: letters + numbers
if (has_letter && has_number) {
  return("Better ")
}

# Rule 3: only letters
if (str_detect(pw, "^[A-Za-z]+$")) {
  return("Meh ")
}

return("Unclear ")
}

# Test cases
check_password("hello")      # Too weak

```

```
[1] "Too weak "
```

```
check_password("hello123")    # Better
```

```
[1] "Better "
```

```
check_password("PassWord!")   # Terrible!
```

```
[1] "Terrible! "
```

```
check_password("abc!23")      # Strong
```

```
[1] "Strong "
```

```
check_password("abcdef")      # Meh
```

```
[1] "Meh "
```

c) **Bonus challenge 2 (you're REALLY bored, right?):** Generalize the Monty Hall Problem to k doors and $k-1$ goats, where the host will open all doors except for 2. Now, visualize the probability of winning (proportion of wins) for both strategies (switch and stay) as a function of the number of simulated rounds n and the number of doors k , for a set of values for both n and k that you choose. *Important:* This is just for fun. You don't have to work on this task if you don't have time. Providing a right/wrong solution does not give/cost you extra points. (A right solution gives you kudos from the instructor though!)

```

# -----
# Task 3 - The Monty Hall Machine
# -----
# This task simulates the classic Monty Hall problem (3 doors)
# and also generalizes it to k doors.
# The goal is to compare the probability of winning when
# staying vs switching doors.
# -----

library(tidyverse)

# -----
# 1 Classic 3-door Monty Hall
# -----
monty_hall_choice <- function(switch = TRUE) {
  doors <- c(1, 2, 3)
  car_door <- sample(doors, 1)          # Randomly hide the car
  contestant_choice <- sample(doors, 1)  # Contestant chooses a door

  # Monty reveals a goat door (not car, not contestant choice)
  available_doors_for_monty <- setdiff(doors, c(contestant_choice, car_door))
  if (contestant_choice == car_door) {
    monty_choice <- sample(available_doors_for_monty, 1)
  } else {
    monty_choice <- available_doors_for_monty
  }

  # If switching, pick the remaining unopened door
  if (switch) {
    contestant_choice <- setdiff(doors, c(contestant_choice, monty_choice))
  }

  # Return TRUE if contestant wins
  contestant_choice == car_door
}

# Example simulation: 100 rounds
set.seed(123)
res_switch <- replicate(100, monty_hall_choice(switch = TRUE))
res_stay <- replicate(100, monty_hall_choice(switch = FALSE))

mean(res_switch) # Probability of winning if switching

```

```
[1] 0.71
```

```
mean(res_stay) # Probability of winning if staying
```

```
[1] 0.32
```

```

# -----
# 2 Simulation wrapper for n rounds
# -----

```

```

monty_hall_simulation <- function(n = 1000, switch = TRUE) {
  wins <- sum(replicate(n, monty_hall_choice(switch = switch)))
  wins / n
}

```

```

# Test wrapper function
monty_hall_simulation(n = 10000, switch = TRUE) # ~0.66

```

```
[1] 0.6576
```

```

monty_hall_simulation(n = 10000, switch = FALSE) # ~0.33

```

```
[1] 0.3291
```

```

# -----
# 3 Generalized Monty Hall (k doors)
# -----
monty_hall_general <- function(k = 3, switch = TRUE) {
  if (k < 3) stop("k must be >= 3 because the host leaves 2 doors closed.")

  doors <- 1:k
  car_door <- sample(doors, 1)
  contestant_choice <- sample(doors, 1)

  # Doors Monty can open (not contestant's choice, not car)
  available_to_open <- setdiff(doors, c(contestant_choice, car_door))

  # Monty keeps exactly one other door closed in addition to contestant's choice
  if (contestant_choice == car_door) {
    monty_keep <- sample(available_to_open, 1)
  } else {
    monty_keep <- car_door
  }

  final_doors <- c(contestant_choice, monty_keep)

  # Switch or stay
  if (switch) {
    contestant_choice <- setdiff(final_doors, contestant_choice)
  }

  # Return TRUE if contestant wins
  contestant_choice == car_door
}

# Wrapper for generalized simulation
monty_hall_general_simulation <- function(n = 1000, k = 3, switch = TRUE) {
  wins <- sum(replicate(n, monty_hall_general(k = k, switch = switch)))
  wins / n
}

# -----

```



```

# 4 Run experiments and visualize
# -----
set.seed(123)
k_values <- c(3, 5, 10, 20, 100)
n_values <- c(100, 1000, 5000)

# Run simulations for all combinations of k, n, switch
results <- expand.grid(k = k_values, n = n_values, switch = c(FALSE, TRUE)) %>%
  mutate(win_prob = mapply(
    function(kk, nn, sw) monty_hall_general_simulation(n = nn, k = kk, switch = sw),
    k, n, switch
  ))

# Analytical probabilities for overlay
theory <- expand.grid(k = k_values, switch = c(FALSE, TRUE), n = n_values) %>%
  mutate(win_prob = ifelse(switch, (k - 1) / k, 1 / k))

# Smooth trend for wider k range
theory_smooth <- expand.grid(k = 3:50, switch = c(FALSE, TRUE), n = n_values) %>%
  mutate(win_prob = ifelse(switch, (k - 1) / k, 1 / k))

# Plot: Simulation vs Analytical
ggplot(results, aes(x = k, y = win_prob, color = switch, group = switch)) +
  geom_line(size = 1.2) +
  geom_point(position = position_dodge(width = 0.2), size = 2) +
  facet_wrap(~ n, labeller = labeller(n = function(x) paste("n =", x))) +

  geom_line(data = theory, aes(x = k, y = win_prob, color = switch, group = switch),
    linetype = "dashed", linewidth = 1.2) +
  geom_point(data = theory, aes(x = k, y = win_prob, color = switch),
    shape = 4, stroke = 1.1, size = 2.2) +

  geom_line(data = theory_smooth, aes(x = k, y = win_prob, color = switch, group = switch),
    linetype = "dotted", linewidth = 0.9) +

  labs(
    title = "Generalized Monty Hall (k doors, host leaves 2 doors closed)",
    subtitle = "Simulation (solid) vs analytical probabilities (dashed = chosen k; dotted = trend)",
    x = "Number of doors (k)",
    y = "Winning probability",
    color = "Strategy"
  ) +
  scale_color_manual(
    breaks = c(FALSE, TRUE),
    labels = c("Stay", "Switch"),
    values = c("red", "blue")
  ) +
  scale_x_continuous(breaks = k_values) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "bottom")

```

Generalized Monty Hall (k doors, host leaves 2 doors closed) Simulation (solid) vs analytical probabilities (dashed = chosen k; do

